

CS1CA Computer Systems Architecture

Processor Pipelining (1)

Week 8 lecture 1 of autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133 in Polly Vacher

Instr No.	Pipeline Stage						
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX
Clock Cycle	1	2	3	4	5	6	7

This topic will be delivered in 4
lectures of weeks 8-9.

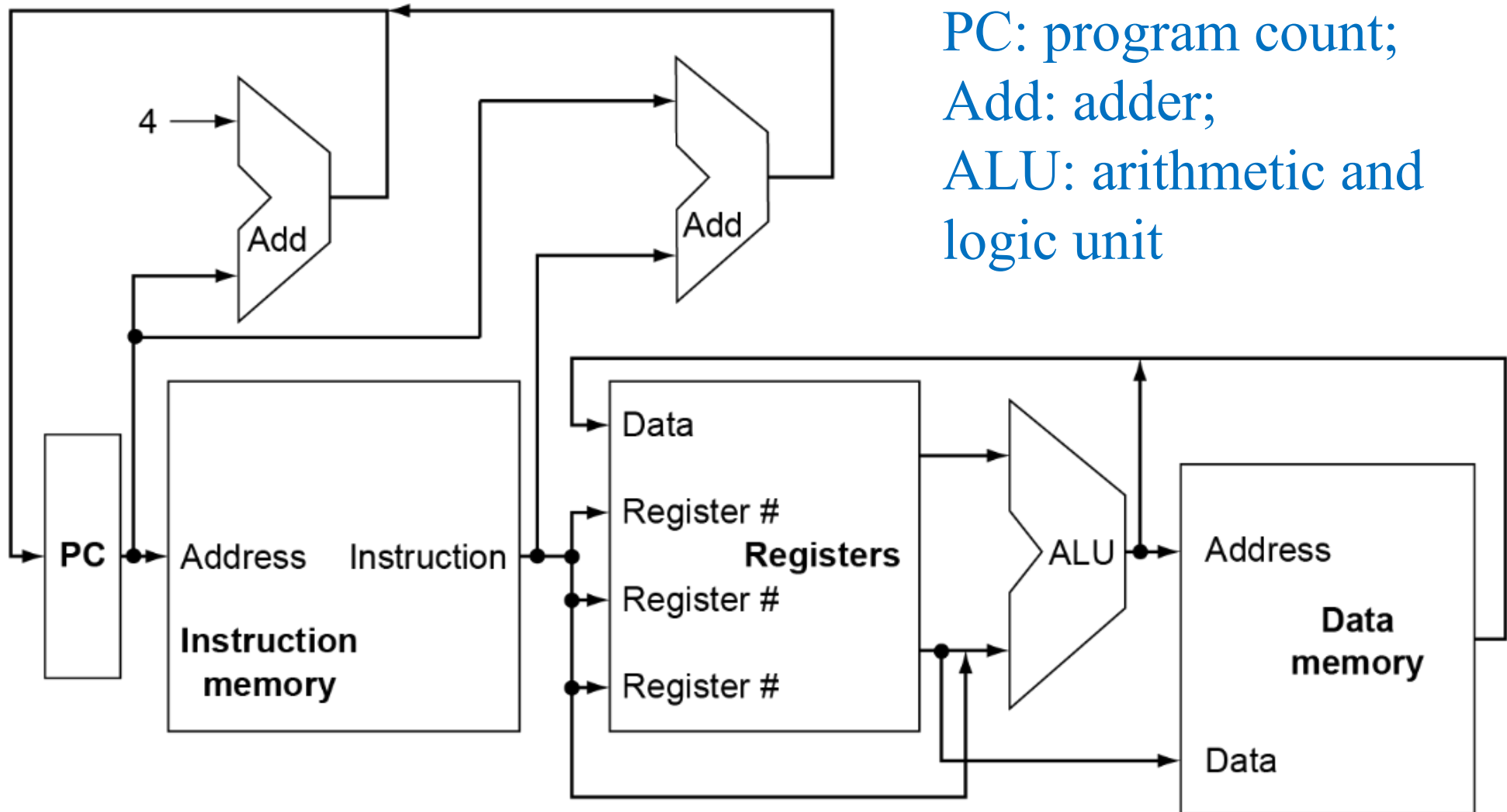
Outline of the topic

- Introduction to pipelining
- The basic pipeline for DLX machine
- Performance issues in pipelining
- Pipeline hazards
 - Data hazards
 - Structure hazards
 - Control hazards
- DLX pipeline to handle multi-cycle operations
- Summary: the MIPS R4000 pipeline (self-learning)

In this lecture

- The following contents will be delivered
 - Introduction to pipelining
 - The basic pipeline for DLX
- The learning outcomes are listed below
 - To gain knowledge of pipelining and its implementation in the typical 5-stage DLX machine
 - To be able to describe pipeline events in each pipeline stage of the 5 stages.

CPU overview associated with memories



PC: program count;
Add: adder;
ALU: arithmetic and
logic unit

Review: instruction execution cycle

- The CPU overview diagram in the previous slide indicates that we can decompose the instruction execution cycle into 5 stages.
 - IF: Instruction fetch from memory (instruction memory)
 - ID: Instruction decode & register read
 - EX: Execute operation or calculate address
 - MEM: Access memory operand (data memory)
 - WB: Write result back to register

What is pipelining?

- Pipelining is a key implementation technique whereby multiple instructions are overlapped in execution.
- It takes advantage of **parallelism** that exists among the actions needed to execute an instruction.
- It is used to make fast CPUs.

Please see an analogy of the pipelined process.

Washing machine analogy

Before the washing machine is pipelined

Time	10 mins	10 mins	10 mins	10 mins	10 mins	10 mins
Users	Pre-wash	Main wash	Dry	Pre-wash	Main wash	Dry

After the washing machine is pipelined

Time (min) user	10 mins	10 mins	10 mins	10 mins	10 mins	10 mins
User 1	Pre-wash	Main wash	Dry			
User 2		Pre-wash	Main wash	Dry		
User 3			Pre-wash	Main wash	Dry	
User 4				Pre-wash	Main wash	Dry

⋮

- Before pipelining, two users complete their washes within an hour.
- After pipelining, four users complete their washes within an hour.
- The machine's performance is improved.

Pipelining – what need to be done

- The idea of washing machine pipelining is applied to processor pipelining.
 - Users are replaced by instructions.
 - Time is clock cycle time.
- The following things need to be done.
 - CPU needs to be modified.
 - Instruction execution needs to be split into sub-processes.
 - Each sub-process takes a part of the instruction execution.

Pipelining – terminology

- Terminology
 - **Pipeline stage**: a sub-process for executing a part of instruction.
 - **Throughput**: the number of instructions per time unit to exit the pipeline.
 - **Processor cycle**: the time required between moving an instruction from one stage to the next stage.
- **Potential speedup = no. of stages in pipeline**

Average execution time per instruction (CPI) on pipelined machine

Pipeline implementation in DLX

- Each DLX instruction can be implemented in at most 5 processor cycles (clock cycles) or 5 stages (the classic five-stage pipeline for a RISC processor), and each cycle does a part of execution.

(1) Instruction fetch cycle (IF)

$\leftarrow \text{IR}$ $\text{Mem}[\text{PC}]$, PC – Program Counter register

~~NPC~~ $\text{PC}+4$, NPC – next PC register

Operation: fetch instruction from the memory address specified by PC and store instruction in the Instruction Register (IR); update value of PC

Example of instruction fetch

- SW 400(R2), R1, where PC=100, I-type format
 - At the IF stage

Instruction
memory

This instruction is in
memory address 100-103.

Operations:

$\text{Reg}(\text{IR}) \leq \text{Mem}[100]$

$\text{NPC} \leq \text{PC} + 4$

0-5	6-10	11-15	16-31	
SW	R2	R1	400	IR

(2) Instruction decode/register fetch cycle (ID)

←A Regs[IR_{6..10}]; IR

SW	R2	R1	400
----	----	----	-----

←B Regs[IR_{11..15}];

Imm Regs[IR_{16..31}] extend 32 bits

Operation:

- i) fetch contents of registers specified in the instruction's fields and store in temporary registers A & B;
- ii) remaining bits are extended to 32 bits (by sign extension) and stored in temporary register Imm

(3) Execution/effective address cycle (EX) - 1

In the 3rd cycle ALU operates on operands obtained in the previous cycle. Different instructions will have different operations.

- Memory reference instruction

ALUOutput $\leftarrow$ A + Imm (I-type instruction)

Operation: ALU adds operands to form the effective address. The result is placed in the Reg ALUOutput.

- Register-register ALU instruction

ALUOutput $\leftarrow$ A *func* B (R-type instruction)

Operation: ALU performs the operation specified by the function code on values in Regs A and B. The result is placed in the Reg ALUOutput.

(3) Execution/effective address cycle (EX) - 2

- **Register-immediate ALU instruction** ALUOutput
 $A \text{ op Imm} \longleftarrow$

Operation: ALU performs the operation specified by the opcode on values in register A and register Imm. The result is placed in the temporary register ALUOutput

- **Branch**

$\text{ALUOutput} \longleftarrow \text{NPC} + \text{Imm};$
 $\text{Cond} \longleftarrow (A \text{ op } 0)$

Operation: 1) ALU adds NPC to the sign-extended immediate value in Imm to complete address of branch target; 2) Reg A (read in previous cycle) is checked to determine whether the branch is taken.

Examples of EX stage events – 1

- For load/store instruction (I-type), **SW 400(R2), R1**
 - $\text{ALUOutput} = \text{A} + \text{Imm} \text{ (Reg[R2] + 400)}$
 - Data memory address is in ALUOutput
- For R-type instruction, **Add R1, R2, R3**

6 5 5 5 11 bits

AL op R2 R3 R1 add R-type format

- At the ID stage, $\text{Reg[R2]} \Rightarrow \text{A}$, $\text{Reg[R3]} \Rightarrow \text{B}$
- $\text{ALUOutput} \leq \text{A} + \text{B}$,

Examples of EX stage events – 2

- For register-immediate ALU instruction (I-type), **ADDI R1, R2, #200**
 - At the ID stage, $\text{Reg}[R2] \Rightarrow A$, $200 \Rightarrow \text{Imm}$
 - $\text{ALUOutput} \leq A + \text{Imm}$
- For branch instruction (I-type), **BEQZ R1, 700**
 - At the ID stage, $\text{Reg}[R1] \Rightarrow A$, $700 \Rightarrow \text{Imm}$
 - $\text{ALUOutput} \leq \text{NPC} + \text{Imm}$
 - Destination memory address is in ALUOutput if taken
 - $\text{Cond} \leq A \text{ op } 0$
 - If True: taken
 - Else (False): untaken

(4) Memory access/branch completion cycle (MEM)

- Memory reference

~~LMD ←~~ Mem[ALUOutput] *or*
~~Mem[ALUOutput] ←~~ B

Operation: 1) Memory load access: data from memory is placed in a temporary Register **LMD**; 2) Memory store access: data from Reg B is written into memory.

- Branch

if (**cond**) ~~PC ←~~ ALUOutput *else* ~~PC ←~~ NPC

Operation: if **cond** is set, PC is replaced with the branch destination address; otherwise, replaced with NPC.

(5) Write-back cycle (WB)

- Register-register ALU instruction

$\text{Regs}[\text{IR}_{16..20}] \leftarrow \text{ALUOutput}$ (R-type)

- Register-immediate ALU instruction

$\text{Regs}[\text{IR}_{11..15}] \leftarrow \text{ALUOutput}$ (I-type)

- Load instruction

$\text{Regs}[\text{IR}_{11..15}] \leftarrow \text{LMD}$ (I-type)

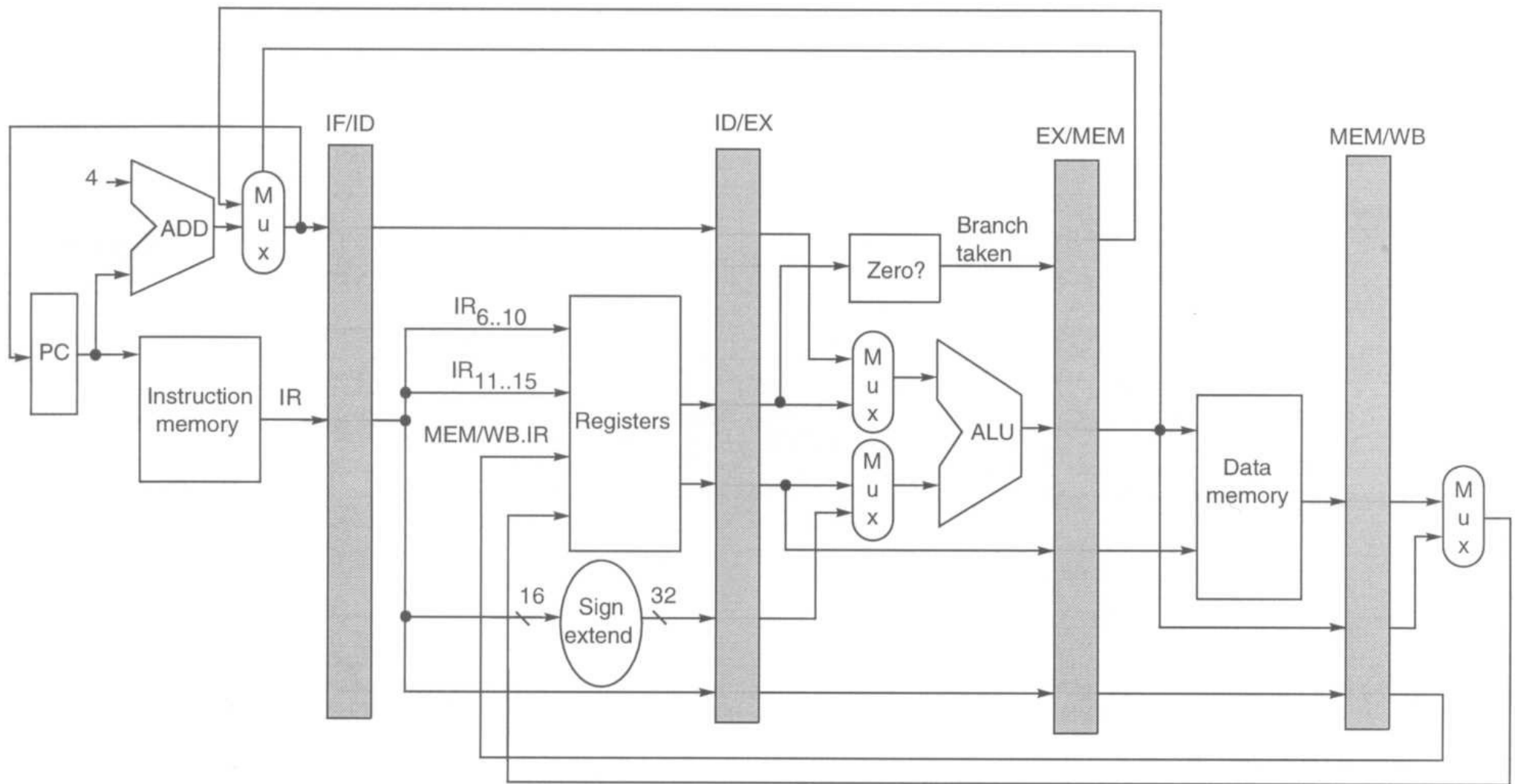
Operation: Write the result into the destination field specified in instruction either from the memory system (LMD) or from the ALU (ALUOutput).

Notes: 1) Branch and store instructions require 4 cycles.
2) All others require 5 cycles.

Pipeline datapath and stages

- A set of registers are added to the pipeline datapath to hold values passed from one pipeline stage to the next (IF/ID, ID/EX, EX/MEM, MEM/WB).
- IF and ID stages are independent from instruction types.
- For branch instructions, the IF stage depends on whether an instruction in EX/MEM is a taken branch (or untaken branch) to fetch the next instruction.
- Different instructions have different operations (or events) at EX stage.
- Control four multiplexors (MUX) to control the pipeline (see the figure on the next slide).

Pipelined datapath by adding registers between pipeline stages



Summary of the lecture

- We have briefly introduced the concept of processor pipelining.
 - Instruction level parallel processing
 - Pipelining reduces average CPI to improve CPU performance
- The DLX pipeline implementation is discussed in detail.
 - Five pipeline stages
 - Each stage takes a part of execution
 - Additional pipeline registers are needed to pass data and instruction from one stage to another.

End of the lecture

Questions and exercises -1

- What is pipelining?
- What are the five stages defined in the DLX pipeline? How is the pipelining implemented in a DLX processor?
- What are pipeline events (or operations) in each pipeline stage?

End of the lecture

Questions and exercises -2

- Describe the events in the five pipeline stages for the following DLX instructions.
 - ADD R1, R2, R3 $\backslash\backslash$ $R2+R3 \Rightarrow R1$
 - ADDI R1, R2, #3 $\backslash\backslash$ $R2+3 \Rightarrow R1$
 - SW 100(R2), R1 $\backslash\backslash$ store Reg[R1] in Mem[[RegR2]+100]
 - BNEZ R1, 100 $\backslash\backslash$ if $R1 \neq 0$, branch to NPC+100
 - LW R1, (R2) $\backslash\backslash$ load R1 with Mem[R2]